

UCS 2312 Data Structures Lab
Assignment 3: Doubly Linked List and its applications

Create an ADT for the doubly linked list data structure with the following functions. Each node which consists of integer data, address of left and right nodes [CO1, K3]

Create a ListADT which has implementations for the following operations

1. Insert an item in the front of the list
void insertFront(listADT L, int c)
2. Insert an item at the end of the list
void insertEnd(listADT L, int c)
3. Insert an item 'd' after the first occurrence 'c' of the list
void insertMiddle(listADT L, int c, int d)
4. Display the items from the list
void displayItems(listADT L)
5. Delete the item present in the list
void deleteItem(listADT L, int c)
6. Search an element in the list and return the number of occurrences
int searchItem(listADT L, int c)

Write a program in C to test the ListADT for its operations with the following test cases.

Testcase:

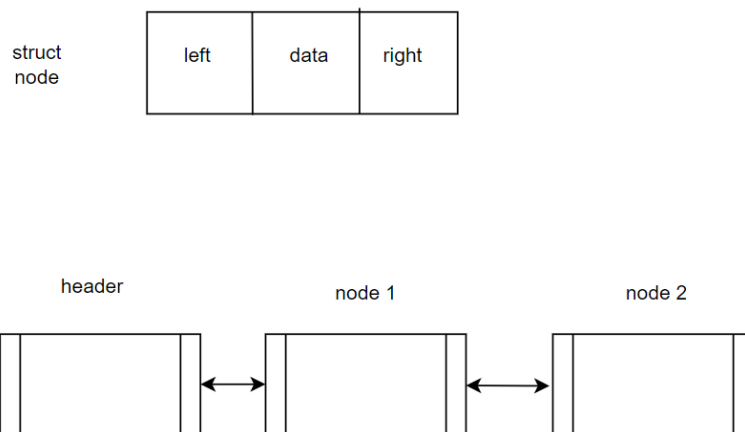
Initially L is Empty

insertFront(L,6) → header ← → 6
insertEnd(L,2) → header ← → 2 ← → 6
insertMiddle(L,2,1) → header ← → 2 ← → 1 ← → 6
insertMiddle(L,2,1) → header ← → 2 ← → 1 ← → 1 ← → 6
search(L,1) → 2

In addition, do the following operations:

1. Check whether the list contains duplicates?
2. Create separate lists containing even and odd numbers from the list
3. Add two 10-digit numbers using the list

DATA STRUCTURE – DOUBLY LINKED LIST



ALGORITHMS:

1. FUNCTION NAME: insertFront

INPUT:

- i) struct node *
- ii) int

OUTPUT: None

```
1. struct node *temp;
2. temp=(struct node *)malloc(sizeof(struct node));
3. temp->data = ele;
4. if (h->right == NULL)
{
    temp->right = h->right;
    h->right = temp;
    temp->left = h;
}
5. else
{
    struct node *ptr;
    ptr=h->right;
    temp->left=h;
    temp->right = ptr;
    ptr->left = temp;
    h->right = temp;
}
```

2.FUNCTION NAME:insertEnd

INPUT:

- i)struct node *
- ii)int

OUTPUT:None

1. struct node *ptr, *prev;
2. ptr = h->right;
3. if (ptr == NULL) {
 struct node *temp;
 temp = (struct node *)malloc(sizeof(struct node));
 temp->data = ele;
 temp->right = h->right;
 temp->left = h;
 h->right = temp;
}
4. else {
 while (ptr != NULL) {
 prev = ptr;
 ptr = ptr->right;
 }
 struct node *temp;
 temp = (struct node *)malloc(sizeof(struct node));
 temp->data = ele;
 temp->right = prev->right;
 prev->right = temp;
 temp->left = prev;
}

3.FUNCTION NAME:insertmiddle

INPUT:

- i)struct node *
- ii)int

OUTPUT:None

1. struct node *ptr;
2. ptr = search(h,key);
3. if (ptr==NULL)
 printf("Key not found!!\n");
4. else if (ptr->right != NULL)
 {
 struct node *temp,*next;
 next = ptr->right;
 temp=(struct node *)malloc(sizeof(struct node));
 temp->data = ele;
 temp->right = ptr->right;

```
        temp->left = next->left;
        next->left = temp;
        ptr->right = temp;
    }
5.  else
    {
        struct node *temp,*next;
        temp=(struct node *)malloc(sizeof(struct node));
        temp->data = ele;
        temp->right = ptr->right;
        ptr->right = temp;
        temp->left=ptr;
    }
```

4.FUNCTION NAME:display

INPUT:

i)struct node *

OUTPUT:None

```
1.  struct node *ptr,*prev;
2.  ptr=h->right;
3.  while(ptr!=NULL)
    {
        printf("%d ",ptr->data);
        prev=ptr;
        ptr=ptr->right;
    }
4.  while (prev->left!=NULL)
    {
        printf("%d ",prev->data);
        prev=prev->left;
    }
```

5.FUNCTION NAME:delete

INPUT:

i)struct node *

ii)int

OUTPUT:None

```
1.  struct node *res;
2.  res=search(h,ele);
3.  if (res==NULL)
        printf("Key not found!!\n");
```

```
4. else if (res->right!=NULL)
{
    struct node *p1,*p2;
    p1=res->left;
    p2=res->right;
    p1->right=p2;
    p2->left=p1;
}
5. else
{
    struct node *p1;
    p1=res->left;
    p1->right=NULL;
}
```

6.FUNCTION NAME:search

INPUT:

i)struct node *

OUTPUT:struct node *

```
1. struct node *ptr;
2. ptr = h->right;
3. while (ptr!=NULL)
{
    if (ptr->data == ele)
        return ptr;
    ptr = ptr->right;
}
4. return NULL;
```

7.FUNCTION NAME:duplicate

INPUT:

i)struct node *

OUTPUT:int

```
1. struct node *ptr,*pt;
2. int dup;
3. ptr = h->right;
4. pt = ptr->right;
5. while(ptr!=NULL)
{
    dup=ptr->data;
    while(pt!=NULL)
    {
```

```
                if (dup==pt->data)
                    return 1;
                else
                    pt=pt->right;
            }
        ptr=ptr->right;
    }
6. return 0;
```

8.FUNCTION NAME:oddeven

INPUT:

- i)struct node *
- ii)struct node *

OUTPUT:None

```
1. struct node *ptr;
2. ptr=h->right;
3. while(ptr!=NULL)
{
    if ((ptr->data)%2==0)
    {insertfront(h1,ptr->data);
    ptr=ptr->right;}
    else
    {insertfront(h2,ptr->data);
    ptr=ptr->right;
    }
}
```

9.FUNCTION NAME:sum

INPUT:

- i)struct node *
- ii)struct node *

OUTPUT:None

```
1. struct node *ptr, *pt;
2. ptr = h->right;
3. int len = 0;
4. while (ptr != NULL) {
    len = len + 1;
    ptr = ptr->right;
}
5. int n = 0;
6. pt = h->right;
```

```
7. while (pt != NULL) {
    len = len - 1;
    n += ((pt->data) * ((int)pow(10, len)));
    pt = pt->right;
}
8. struct node *ptr1, *pt1;
9. ptr1 = h1->right;
10. int len1 = 0;
11. while (ptr1 != NULL) {
    len1 = len1 + 1;
    ptr1 = ptr1->right;
}
12. int n1 = 0;
13. pt1 = h1->right;
14. while (pt1 != NULL) {
    len1 = len1 - 1;
    n1 += ((pt1->data) * ((int)pow(10, len1)));
    pt1 = pt1->right;
}
15. int sum = n1+n;

16. struct node *h2;
17. h2=(struct node *)malloc(sizeof(struct node));
18. h2->left=h2->right=NULL;
19. while (sum > 0)
    {
        insertfront (h2, sum % 10);
        sum = sum / 10;
    }
display(h2);
```

CODE:
listadt.h

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
struct node
{
    int data;
    struct node *left;
    struct node *right;
};
struct node* search(struct node *h,int ele)
{
    struct node *ptr;
    ptr = h->right;
```

```
while (ptr!=NULL)
{
    if (ptr->data == ele)
        return ptr;
    ptr = ptr->right;
}
return NULL;
}
```

```
void insertfront(struct node *h,int ele)
{
    struct node *temp;
    temp=(struct node *)malloc(sizeof(struct node));
    temp->data = ele;
    if (h->right == NULL)
    {
        temp->right = h->right;
        h->right = temp;
        temp->left = h;
    }
    else
    {
        struct node *ptr;
        ptr=h->right;
        temp->left=h;
        temp->right = ptr;
        ptr->left = temp;
        h->right = temp;
    }
}
```

```
void insertend(struct node *h, int ele) {

    struct node *ptr, *prev;
    ptr = h->right;
    if (ptr == NULL) {
        struct node *temp;
        temp = (struct node *)malloc(sizeof(struct node));
        temp->data = ele;
        temp->right = h->right;
        temp->left = h;
        h->right = temp;
    } else {
        while (ptr != NULL) {
            prev = ptr;
            ptr = ptr->right;
        }
    }
}
```



```
    struct node *temp;
    temp = (struct node *)malloc(sizeof(struct node));
    temp->data = ele;
    temp->right = prev->right;
    prev->right = temp;
    temp->left = prev;
}
}
```

```
void insertmiddle(struct node *h,int key,int ele)
{
    struct node *ptr;
    ptr = search(h,key);
    if (ptr==NULL)
        printf("Key not found!!\n");
    else if (ptr->right != NULL)
    {
        struct node *temp,*next;
        next = ptr->right;
        temp=(struct node *)malloc(sizeof(struct node));
        temp->data = ele;
        temp->right = ptr->right;
        temp->left = next->left;
        next->left = temp;
        ptr->right = temp;
    }
    else
    {
        struct node *temp,*next;
        temp=(struct node *)malloc(sizeof(struct node));
        temp->data = ele;
        temp->right = ptr->right;
        ptr->right = temp;
        temp->left=ptr;
    }
}
```

```
void display(struct node *h)
{
    struct node *ptr,*prev;
    ptr=h->right;
    printf("FORWARD :");
    while(ptr!=NULL)
    {
        printf("%d ",ptr->data);
        prev=ptr;
        ptr=ptr->right;
    }
}
```

```
    }  
    printf("\n");  
    printf("BACKWARD:");  
    while (prev->left!=NULL)  
    {  
        printf("%d ",prev->data);  
        prev=prev->left;  
    }  
}
```

```
void delete(struct node *h,int ele)
```

```
{  
    struct node *res;  
    res=search(h,ele);  
    if (res==NULL)  
        printf("Key not found!!\n");  
    else if (res->right!=NULL)  
    {  
        struct node *p1,*p2;  
        p1=res->left;  
        p2=res->right;  
        p1->right=p2;  
        p2->left=p1;  
    }  
    else  
    {  
        struct node *p1;  
        p1=res->left;  
        p1->right=NULL;  
    }  
}
```

```
int duplicate(struct node *h)
```

```
{  
    struct node *ptr,*pt;  
    int dup;  
    ptr = h->right;  
    pt = ptr->right;  
    while(ptr!=NULL)  
    {  
        dup=ptr->data;  
        while(pt!=NULL)  
        {  
            if (dup==pt->data)  
                return 1;  
            else
```

```
        pt=pt->right;
    }
    ptr=ptr->right;
}
return 0;
}
```

```
void evenodd(struct node *h,struct node *h1,struct node *h2)
```

```
{
    struct node *ptr;
    ptr=h->right;
    while(ptr!=NULL)
    {
        if ((ptr->data)%2==0)
        {insertfront(h1,ptr->data);
          ptr=ptr->right;}
        else
        {insertfront(h2,ptr->data);
          ptr=ptr->right;
        }
    }
}
```

```
void posneg(struct node *h,struct node *h1,struct node *h2)
```

```
{
    struct node *ptr;
    ptr=h->right;
    while(ptr!=NULL)
    {
        if ((ptr->data>0))
        {insertfront(h1,ptr->data);
          ptr=ptr->right;}
        else
        {insertfront(h2,ptr->data);
          ptr=ptr->right;
        }
    }
}
```

```
void add(struct node *h, struct node *h1) {
```

```
    struct node *ptr, *pt;
    ptr = h->right;
    int len = 0;
    while (ptr != NULL) {
        len = len + 1;
        ptr = ptr->right;
    }
    //printf("%d \n", len);
```

```
int n = 0;
pt = h->right;
while (pt != NULL) {
    len = len - 1;
    n += ((pt->data) * ((int)pow(10, len)));
    pt = pt->right;
}
struct node *ptr1, *pt1;
ptr1 = h1->right;
int len1 = 0;
while (ptr1 != NULL) {
    len1 = len1 + 1;
    ptr1 = ptr1->right;
}
//printf("%d \n", len);
int n1 = 0;
pt1 = h1->right;
while (pt1 != NULL) {
    len1 = len1 - 1;
    n1 += ((pt1->data) * ((int)pow(10, len1)));
    pt1 = pt1->right;
}
printf("%d %d\n", n, n1);
int sum = n1+n;
int temp=sum;

struct node *h2;
h2=(struct node *)malloc(sizeof(struct node));
h2->left=h2->right=NULL;
while (sum > 0)
{
    insertfront(h2, sum % 10);
    sum = sum / 10;
}
display(h2);
}

int palindrome(struct node *h, int num) {
    int temp = num;
    int rev = 0;
    int i = 0;

    while (temp > 0) {
        rev = (rev * 10) + (temp % 10);
        temp = temp / 10;
        i = i + 1;
    }
}
```

```
if (rev==num)
    return 1;
else
    return 0;
}
```

```
int findmiss(struct node *h,int k)
{
    struct node *ptr;
    ptr=h->right;
    int k1=0;
    for (int i=1;i<25;i++)
    {
        if (ptr->right!=NULL)
        {
            if ((i!=ptr->data)&&(i<ptr->data))
            {
                k1++;
                if (k1==k)
                    return i;
            }
        }
        else
        {
            ptr=ptr->right;
        }
    }
    else
    {
        if (k1==k)
            return i;
        else
            k1+=1;
    }
}
```

```
int divisor(struct node *h)
{
    struct node *ptr;
    int i=1;
    int k =1;
    while(k=1)
    {
        int sum=0;
        ptr=h->right;
        while(ptr!=NULL)
        {
```

```
    sum+=ceil((ptr->data)/i);  
    ptr=ptr->right;  
}  
if (sum<=8)  
{  
    break;  
}  
else  
    {i+=1;}  
}  
return i;  
}
```

Main.c

```
#include <stdio.h>  
#include <stdlib.h>  
#include "listadt.h"
```

```
void main()  
{  
    struct node *header;  
    header = (struct node *)malloc(sizeof(struct node));  
    header->left = NULL;  
    header->right = NULL;  
    printf("----INSERT FRONT---\n");  
    printf("enter the number of elements :");  
    int n;  
    scanf("%d",&n);  
    for (int i = 0;i<n;i++)  
    {  
        printf("ELEMENT : %d\n",i+1));  
        printf("enter the element :");  
        int a;  
        scanf("%d",&a);  
        insertfront(header,a);  
    }  
    printf("---DISPLAY---\n");  
    display(header);  
    printf("\n");  
  
    printf("---LEAST COMMON DIVISOR--\n");  
    printf("Least common divisor is : %d \n",divisor(header));  
    struct node *h1,*h2;  
    h1=(struct node *)malloc(sizeof(struct node));  
    h1->left=h1->right=NULL;  
    h2=(struct node *)malloc(sizeof(struct node));  
    h2->left=h2->right=NULL;
```

```
posneg(header,h1,h2);
printf("\n--POSITIVE----\n ");
display(h1);
printf("\n--NEGATIVE----\n ");
display(h2);
printf("\n");

printf("---FINDING MISSING ELEMENT ---\n");
printf("enter which missing element you want to find :");
int k;
scanf("%d",&k);
printf("MISSING ELEMENT : %d",findmiss(header,k));
printf("----INSERT END---\n");
printf("enter the number of elements :");
int n1;
scanf("%d",&n1);
for (int i = 0;i<n1;i++)
{
    printf("ELEMENT : %d\n",(i+1));
    printf("enter the element :");
    int a;
    scanf("%d",&a);
    insertend(header,a);
}
printf("---DISPLAY---\n");
display(header);
printf("\n");
printf("---INSERT MIDDLE----\n");
printf("enter the number of elements to be inserted in middle:");
int n2;
scanf("%d",&n2);
for (int i = 0;i<n2;i++)
{
    printf("ELEMENT : %d\n",(i+1));
    printf("enter the element :");
    int a;
    scanf("%d",&a);
    printf("enter the key after which the element will be inserted:");
    int ah;
    scanf("%d",&ah);
    insertmiddle(header,ah,a);
}
printf("---DISPLAY---\n");
display(header);
printf("\n");
printf("----SEARCH----\n");
struct node *re;
```

```
printf("enter the element to be searched:");
int elt;
scanf("%d",&elt);
re=search(header,elt);
if (re==NULL)
    printf("NOT FOUND!\n");
else
    printf("FOUND!!\n");
printf("--DELETION--\n");
printf("enter the element to be deleted :\n");
int a;
scanf("%d",&a);
delete(header,a);
printf("\n---DISPLAY---\n");
display(header);
printf("\n");
printf("---DUPLICATES ----\n");
int res = duplicate(header);
if (res==0)
    printf("NO DUPLICATES\n");
else
    printf("DUPLICATES FOUND\n");

printf("---EVEN AND ODD---");
struct node *h11,*h22;
h11=(struct node *)malloc(sizeof(struct node));
h11->left=NULL;
h11->right=NULL;
h22=(struct node *)malloc(sizeof(struct node));
h22->left=NULL;
h22->right=NULL;
evenodd(header,h11,h22);
printf("\n---DISPLAY---\n");
display(header);
printf("\nEVEN LIST:\n");
display(h11);
printf("\nODD LIST:\n");
display(h22);
printf("\n");
printf("\n---ADDITION---\n");
printf("enter the number to be added :");
int num;
scanf("%d",&num);
int result = add(header,num);
printf("\n--PALINDROME---\n");
int res1 = palindrome(header,result);
if (res1==1)
```



```
    printf("YES!");  
else  
    printf("NO!");  
}
```

OUTPUT:

```
C:\Users\User\OneDrive\SEM3\DATA STRUCTURES\LAB\DOUBLY_LINKED_LIST  
----INSERT FRONT---  
enter the number of elements :4  
ELEMENT : 1  
enter the element :4  
ELEMENT : 2  
enter the element :3  
ELEMENT : 3  
enter the element :2  
ELEMENT : 4  
enter the element :1  
---DISPLAY---  
FORWARD :1 2 3 4  
BACKWARD:4 3 2 1  
  
----INSERT END---  
enter the number of elements :3  
ELEMENT : 1  
enter the element :5  
ELEMENT : 2  
enter the element :6  
ELEMENT : 3  
enter the element :7  
---DISPLAY---  
FORWARD :1 2 3 4 5 6 7  
BACKWARD:7 6 5 4 3 2 1  
----INSERT MIDDLE----  
enter the number of elements to be inserted in middle:1  
ELEMENT : 1  
enter the element :16  
enter the key after which the element will be inserted:4
```

```
---DISPLAY---  
FORWARD :1 2 3 4 16 5 6 7  
BACKWARD:7 6 5 16 4 3 2 1  
----SEARCH----  
enter the element to be searched:16  
FOUND!!  
--DELETION--  
enter the element to be deleted :  
2  
  
---DISPLAY---  
FORWARD :1 3 4 16 5 6 7  
BACKWARD:7 6 5 16 4 3 1  
---DUPLICATES ----  
NO DUPLICATES  
---EVEN AND ODD---  
---DISPLAY---  
FORWARD :1 3 4 16 5 6 7  
BACKWARD:7 6 5 16 4 3 1  
EVEN LIST:  
FORWARD :6 16 4  
BACKWARD:4 16 6  
ODD LIST:  
FORWARD :7 5 3 1  
BACKWARD:1 3 5 7
```

```
----INSERT END---
enter the number of elements :10
ELEMENT : 1
enter the element :1
ELEMENT : 2
enter the element :1
ELEMENT : 3
enter the element :1
ELEMENT : 4
enter the element :1
ELEMENT : 5
enter the element :1
ELEMENT : 6
enter the element :1
ELEMENT : 7
enter the element :1
ELEMENT : 8
enter the element :1
ELEMENT : 9
enter the element :1
ELEMENT : 10
enter the element :1
---DISPLAY---
FORWARD :1 1 1 1 1 1 1 1 1 1
BACKWARD:1 1 1 1 1 1 1 1 1 1
```

```
----INSERT END---
enter the number of elements :10
ELEMENT : 1
enter the element :3
ELEMENT : 2
enter the element :3
ELEMENT : 3
enter the element :3
ELEMENT : 4
enter the element :3
ELEMENT : 5
enter the element :3
ELEMENT : 6
enter the element :3
ELEMENT : 7
enter the element :3
ELEMENT : 8
enter the element :3
ELEMENT : 9
enter the element :3
ELEMENT : 10
enter the element :3
FORWARD :3 3 3 3 3 3 3 3 3 3
BACKWARD:3 3 3 3 3 3 3 3 3 3
FORWARD :1 4 9 4 7 7 1 4 8
BACKWARD:8 4 1 7 7 4 9 4 1
```

QUESTION 2: Print the k-th missing element

CODE:

```
int findmiss(struct node *h,int k)
{
    struct node *ptr;
    ptr=h->right;
    int k1=0;
    for (int i=1;i<25;i++)
    {
        if (ptr->right!=NULL)
        {
            if ((i!=ptr->data)&&(i<ptr->data))
            {
                k1++;
                if (k1==k)
                    return i;
            }
            else
            {
                ptr=ptr->right;
            }
        }
        else
        {
            if (k1==k)
                return i;
            else
                k1+=1;
        }
    }
}
```

OUTPUT:

```
----INSERT FRONT---
enter the number of elements :4
ELEMENT : 1
enter the element :8
ELEMENT : 2
enter the element :5
ELEMENT : 3
enter the element :3
ELEMENT : 4
enter the element :1
---DISPLAY---
FORWARD :1 3 5 8
BACKWARD:8 5 3 1
```

```
---FINDING MISSING ELEMENT ---
enter which missing element you want to find :2
MISSING ELEMENT : 4
```

QUESTION 3: Given a linked list, find the least common number when divided by with each element yields a sum less than 8.

CODE:

```
int divisor(struct node *h)
{
    struct node *ptr;
    int i=1;
    int k=1;
    while(k=1)
    {
        int sum=0;
        ptr=h->right;
        while(ptr!=NULL)
        {
            sum+=ceil((ptr->data)/i);
            ptr=ptr->right;
        }
        if (sum<=8)
        {
            break;
        }
        else
            {i+=1;}
    }
    return i;
}
```

OUTPUT:

```
----INSERT FRONT---
enter the number of elements :4
ELEMENT : 1
enter the element :8
ELEMENT : 2
enter the element :5
ELEMENT : 3
enter the element :3
ELEMENT : 4
enter the element :1
---DISPLAY---
FORWARD :1 3 5 8
BACKWARD:8 5 3 1
---LEAST COMMON DIVISOR--
Least common divisor is : 2
```